

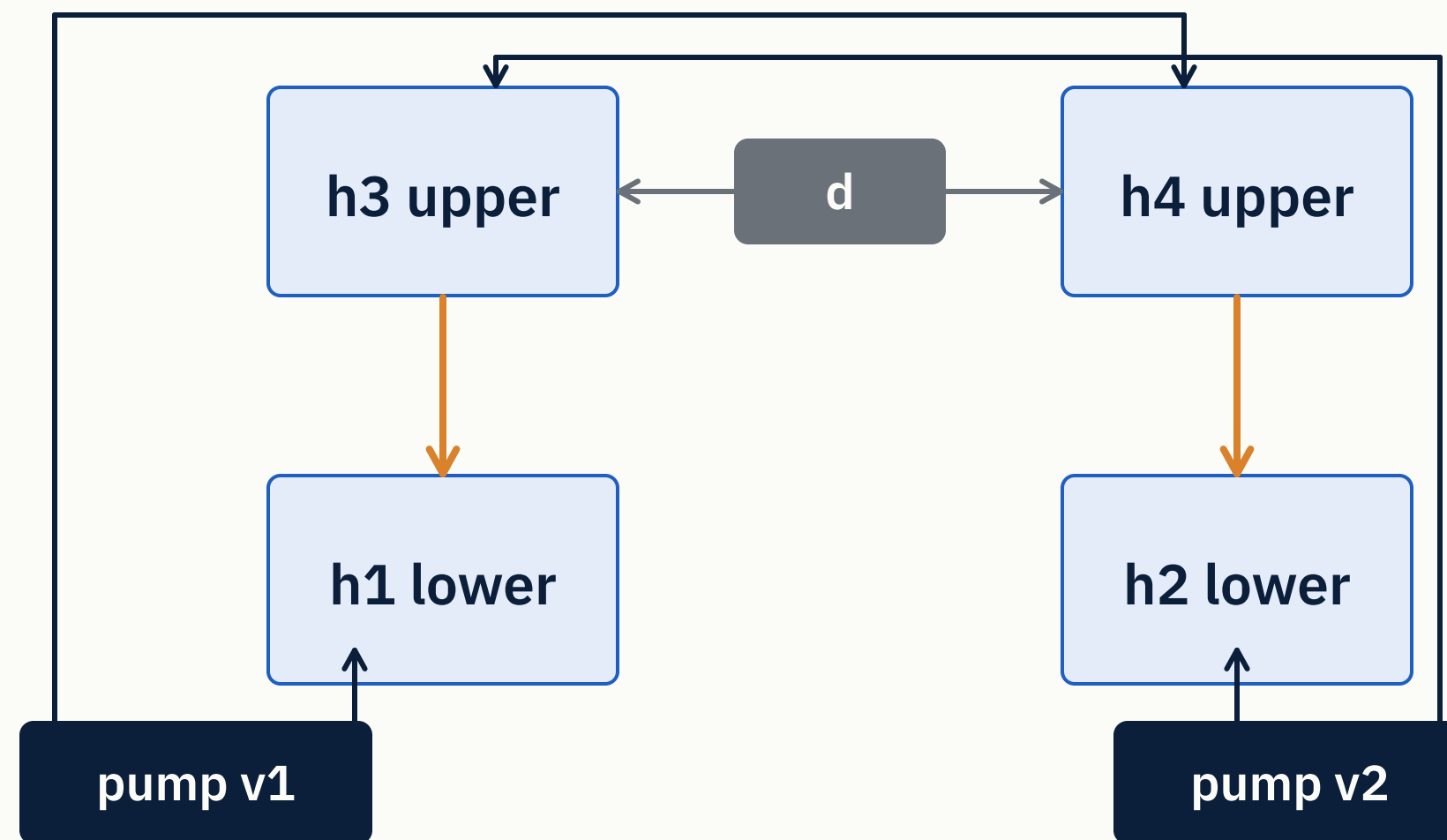
Which causal-discovery algorithm recovers the quadruple-tank wiring from data alone?

PC, PCMCI+ and VAR-LiNGAM tested on a simulated four-tank process with a known answer key: 7 signals, 8 true links, two operating points.

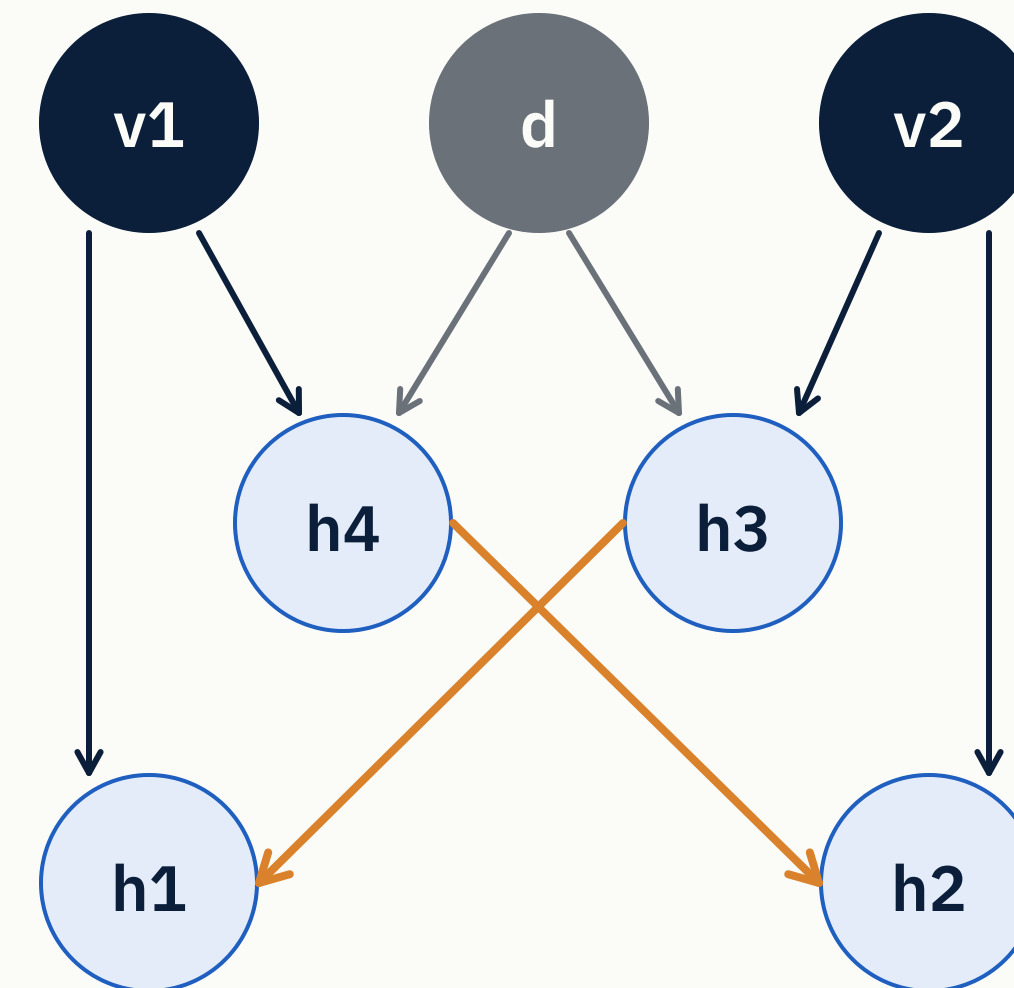
THE SYSTEM

Two pumps, four tanks, eight pipes: the graph on the right is the answer key for every result that follows

Physical rig (Johansson quadruple-tank process)



Same system as a causal graph: 7 nodes, 8 arrows



Pumps split between a lower and an upper tank; upper tanks drain into the tank below (orange = the two drainage links, $h3 \rightarrow h1$ and $h4 \rightarrow h2$); d feeds both upper tanks.

THE GROUND TRUTH

Every arrow on the previous slide is one term in the four mass-balance equations of the simulator

Level h_i in cm, pump voltages v_1, v_2 in V, disturbance flow d , $g = 981 \text{ cm/s}^2$. A_i tank area, a_i outlet area, γ valve split, k_i pump gain.

$$\dot{h}_1 = -(a_1/A_1)\sqrt{(2g h_1)} + (a_3/A_1)\sqrt{(2g h_3)} + (\gamma_1 k_1/A_1) v_1$$

$$\dot{h}_2 = -(a_2/A_2)\sqrt{(2g h_2)} + (a_4/A_2)\sqrt{(2g h_4)} + (\gamma_2 k_2/A_2) v_2$$

$$\dot{h}_3 = -(a_3/A_3)\sqrt{(2g h_3)} + ((1-\gamma_2)k_2/A_3) v_2 + (\gamma_3/A_3) d$$

$$\dot{h}_4 = -(a_4/A_4)\sqrt{(2g h_4)} + ((1-\gamma_1)k_1/A_4) v_1 + ((1-\gamma_3)/A_4) d$$

Blue terms are the four pump arrows. Each pump appears in two equations because the valve splits its flow by γ .

Orange terms are the two drainage arrows: the upper tank's outflow $\sqrt{(2g h_3)}$ is an inflow to the lower tank.

Grey terms are the disturbance arrows, split $\gamma_3 = 0.4$ between the upper tanks.

The first term in each row is the tank's own outflow: the self-dependence $h_i(t-1) \rightarrow h_i(t)$ that every time-series method must handle.

Parameters: $A_1, A_3 = 28 \text{ cm}^2$, $A_2, A_4 = 32 \text{ cm}^2$, $a_1, a_3 = 0.071$, $a_2, a_4 = 0.057 \text{ cm}^2$, $k_1 = 3.14$, $k_2 = 3.29$, $\gamma_1 = 0.43$, $\gamma_2 = 0.34$. Measurement and input noise $\sigma = 0.05$.

THE DATASETS

Two datasets because the same rig behaves two different ways depending on how the valves split the flow

Same 8 true arrows and same inputs in both: 4 h at 5 s sampling, 2880 rows each. Only the strength and timing of each arrow changes.

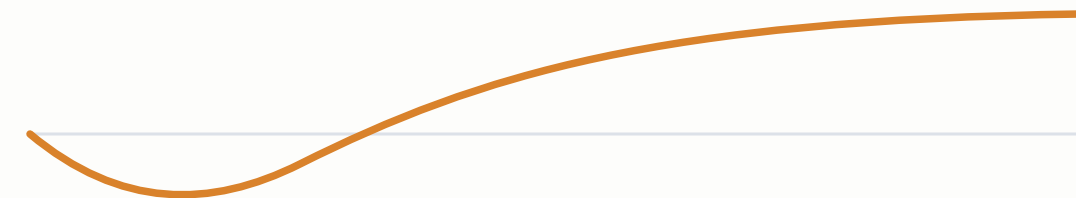
P- · minimum phase · the "easy" control case



Lower-tank level after a pump step: rises promptly

- Most of each pump's flow goes straight to the lower tanks
- Drainage links carry the smaller share of water
- Time constants 23–91 s

P+ · non-minimum phase · the "hard" control case



Lower-tank level after a pump step: briefly moves the wrong way, then follows

- Most of each pump's flow goes to the upper tanks first
- Drainage links carry the larger share of water
- Time constants 39–91 s

THE DATA

Inputs are square waves and strongly non-Gaussian; the tank levels are smooth, near-Gaussian and highly autocorrelated

Measured on the generated data before any algorithm ran. These properties decide which method assumptions hold.

SHAPE OF THE DISTRIBUTION

-2.0

excess kurtosis of v_1, v_2, d : a two-level square wave, as far from Gaussian as it gets (Jarque–Bera $p \approx 10^{-105}$)

Levels h_1-h_4 : Jarque–Bera p between 0.008 and 0.35, close to Gaussian because each tank low-pass filters its inputs

MEMORY

0.93–0.99

lag-1 autocorrelation of the levels; inputs 0.83–0.90

Each sample is almost a copy of the previous one. A method that ignores this sees spurious links everywhere.

STATIONARITY AND SAMPLING

$p < 10^{-4}$

augmented Dickey–Fuller test on all 7 columns: no unit root, no trend

$T_s = 5$ s, 4 h, 2880 rows per dataset, 600 s burn-in discarded. Pumps switch every 30 s, disturbance every 50 s.

Consequence: partial correlation (linear) is defensible for the levels; the non-Gaussian pump signals are what VAR-LiNGAM's direction rule needs, and they are exactly the variables that are inputs, not tanks.

HOW WE SCORE

Every algorithm hands back a graph; we count how many of the 8 true arrows it found, missed, flipped or invented

EDGE OUTCOMES

Found: true arrow, right direction (TP)

Missed: true arrow not reported (FN)

Reversed: true pair, wrong direction

Invented: reported arrow with no pipe behind it (FP)

Lagged edges are collapsed to one arrow per pair before scoring. Self-loops are counted separately.

TWO SCORES, READ TOGETHER

$\text{precision} = \text{TP} / (\text{TP} + \text{FP} + \text{reversed})$

$\text{recall} = \text{TP} / 8$

$\text{F1} = 2 \cdot \text{precision} \cdot \text{recall} / (\text{precision} + \text{recall})$

SHD = missed + invented + reversed: the number of single-edge edits to reach the true graph. Lower is better; 0 is perfect.

F1 already punishes invented edges through precision; SHD adds a raw count that does not saturate and treats a flip as one edit, not two.

SHARED SETTINGS, MAIN RUN

Rows: every 3rd sample, 960 rows per dataset at $T_s = 15$ s

Window: $\tau_{\max} = 20$ steps = 300 s, about 3 time constants of the slowest tank

Test: partial correlation, $\alpha = 0.01$

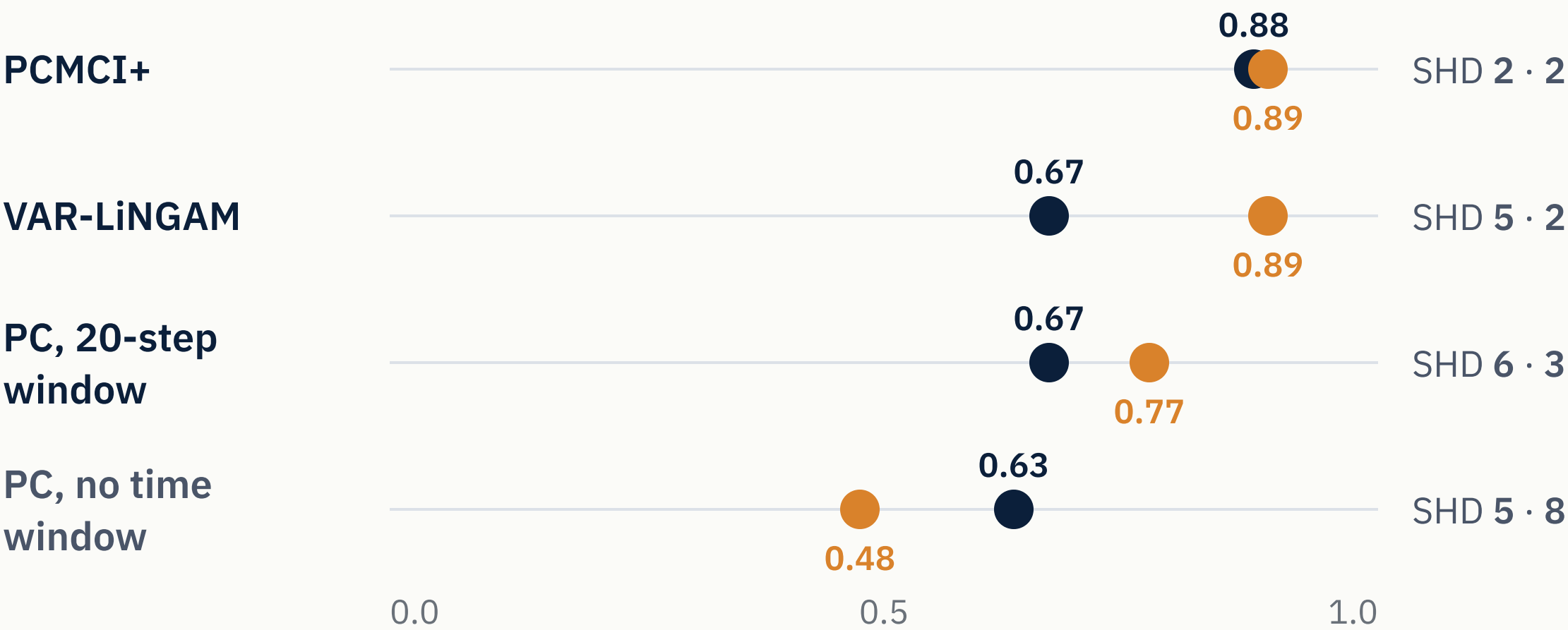
VAR-LINGAM: 20 lags, 100 bootstraps, keep edge if in ≥ 90

The same three methods were also run on all 2880 rows at 5 s (100 s window); that comparison is on its own slide.

EXECUTIVE SUMMARY

PCMCI+ recovers the wiring best at both operating points; the two drainage links cause almost every error

F1 on the 8 true links (1.0 = perfect), with SHD (edits to reach the truth) beside each row. ● P- ● P+



15 of 16
true links found by PCMCI+ across both datasets, none reversed

1.5 s vs 134 s
runtime per dataset, PCMCI+ vs VAR-LiNGAM

2 arrows
the drainage links h3→h1, h4→h2 account for 7 of the 10 misses and flips across all six runs

THE METHODS

Three ways of asking "does A drive B?", each with a different blind spot

PC

Removes a link whenever two signals become unrelated once you control for the others.

Built for snapshots, not time series. We ran it twice: with no time window, and with the same 20-step window as the others.

Blind spot: no correction for signals that mostly predict themselves

PCMCI+

PC redesigned for time series. Tests each link at every lag in the window and corrects for autocorrelation.

The method the literature recommends for slow, autocorrelated process data. Our pre-registered favourite.

Blind spot: linear test only, in this run

VAR-LINGAM

Fits a linear forecasting model, then uses the shape of the noise (non-Gaussian) to decide which way each link points.

Keeps a link only if it survives 90 of 100 bootstrap resamples.

Blind spot: direction is unreliable where the noise is Gaussian (the tank levels)

PC deletes a link whenever a conditional-independence test says two variables are unrelated given some others

Step 1 · Skeleton

Start fully connected. For every pair (X, Y) and conditioning set S of growing size, remove the edge if

$$X \perp Y \mid S \Leftrightarrow \rho(X, Y \mid S) = 0$$

$\rho(X, Y \mid S)$ is the partial correlation: regress X and Y on S, correlate the residuals. Tested with Fisher's z:

$$z = \frac{1}{2} \ln[(1+\rho)/(1-\rho)] \cdot \sqrt{(n - |S| - 3)} \sim N(0,1)$$

Edge removed if the two-sided p-value exceeds $\alpha = 0.01$. The S that separated them is remembered.

Step 2 · Orientation

For every path $X - Z - Y$ with X and Y not adjacent: if Z was *not* in the set that separated X and Y, orient as a collider

$$X \rightarrow Z \leftarrow Y$$

Then propagate with Meek's rules (no new colliders, no cycles). Edges neither rule reaches stay undirected.

How we ran it here

No window: the 7 variables at the same time step only. **20-step window:** the lagged copies $X_{t-1} \dots X_{t-20}$ are added as extra variables and time order is used to forbid future $\rightarrow$ past. No correction for autocorrelation in either case.

PCMCI+ runs the same tests on time-lagged variables, but conditions on both variables' own past so autocorrelation cannot fake a link

Step 1 · Lagged parent search (PC_l)

For each target X_{i_t} , candidates are every $X_{t-\tau}^j$ with $1 \leq \tau \leq \tau_{\max} = 20$. Iteratively drop a candidate when

$$X_{t-\tau}^i \perp X_t^j \mid S, S \subseteq \text{strongest remaining candidates}$$

Output: an estimated parent set $\hat{P}(X_{i_t})$ for every variable.
Contemporaneous links ($\tau = 0$) are handled in the same loop with S drawn from both lagged and same-time parents.

Cost: every (i, j, τ) triple is tested, $7 \times 7 \times 21 \approx 1000$ pairs plus conditioning sets, but each ParCorr test is a small regression. Both datasets finished in 1.5 s.

Step 2 · Momentary conditional independence (MCI)

Every surviving link is re-tested with both parent sets in the condition:

$$X_{t-\tau}^i \perp X_t^j \mid \hat{P}(X_t^j) \setminus \{X_{t-\tau}^i\}, \hat{P}(X_{t-\tau}^i)$$

Conditioning on $\hat{P}(X_{t-\tau}^i)$ removes the cause's own memory; conditioning on $\hat{P}(X_t^j)$ removes the effect's. What is left is the momentary effect of i on j , and its false-positive rate is controlled at α even under strong autocorrelation.

Same-time links are oriented by collider rules; lagged links by time order. Same ParCorr test, $\alpha = 0.01$.

VAR-LiNGAM fits a linear forecasting model, then uses the non-Gaussian shape of the residuals to decide which way each same-time link points

The model

$$\mathbf{x}_t = \mathbf{B}_0 \mathbf{x}_t + \sum_{\tau=1..p} \mathbf{B}_\tau \mathbf{x}_{t-\tau} + \mathbf{e}_t$$

$\mathbf{B}_0$ is the same-time graph (must be acyclic), $\mathbf{B}_\tau$ the lagged effects, $\mathbf{e}_t$ independent and **non-Gaussian**. $p = 20$ here.

Step 1 · Ordinary VAR

$$\mathbf{x}_t = \sum \mathbf{M}_\tau \mathbf{x}_{t-\tau} + \mathbf{n}_t \text{ (least squares)}$$

The residuals $\mathbf{n}_t$ contain all the same-time structure, mixed together.

Step 2 · LiNGAM on the residuals

$$\mathbf{n}_t = \mathbf{B}_0 \mathbf{n}_t + \mathbf{e}_t$$

DirectLiNGAM finds the causal order: the variable whose regression residual is most *independent* of the others is the root; regress it out, repeat. Independence, not correlation, is what breaks the symmetry, and it needs non-Gaussian $\mathbf{e}_t$. With Gaussian noise every order fits equally well.

Step 3 · Recover the lagged effects

$$\mathbf{B}_\tau = (\mathbf{I} - \mathbf{B}_0) \mathbf{M}_\tau$$

Bootstrap: refit on 100 resampled series; keep an edge only if non-zero in 90 of them. That is where the 134 s goes.

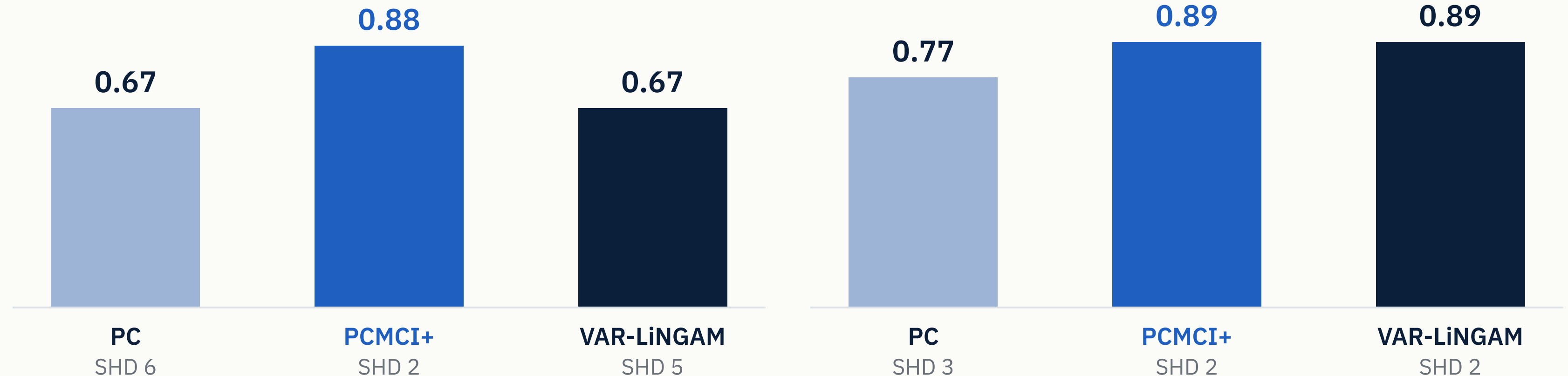
HEADLINE RESULT

PCMCI+ recovers 7 to 8 of the 8 true links with at most 2 mistakes at either operating point

F1 on 8 directed edges; SHD under each bar. 960 rows at 15 s, 300 s window, partial correlation, $\alpha = 0.01$.

P- · minimum phase

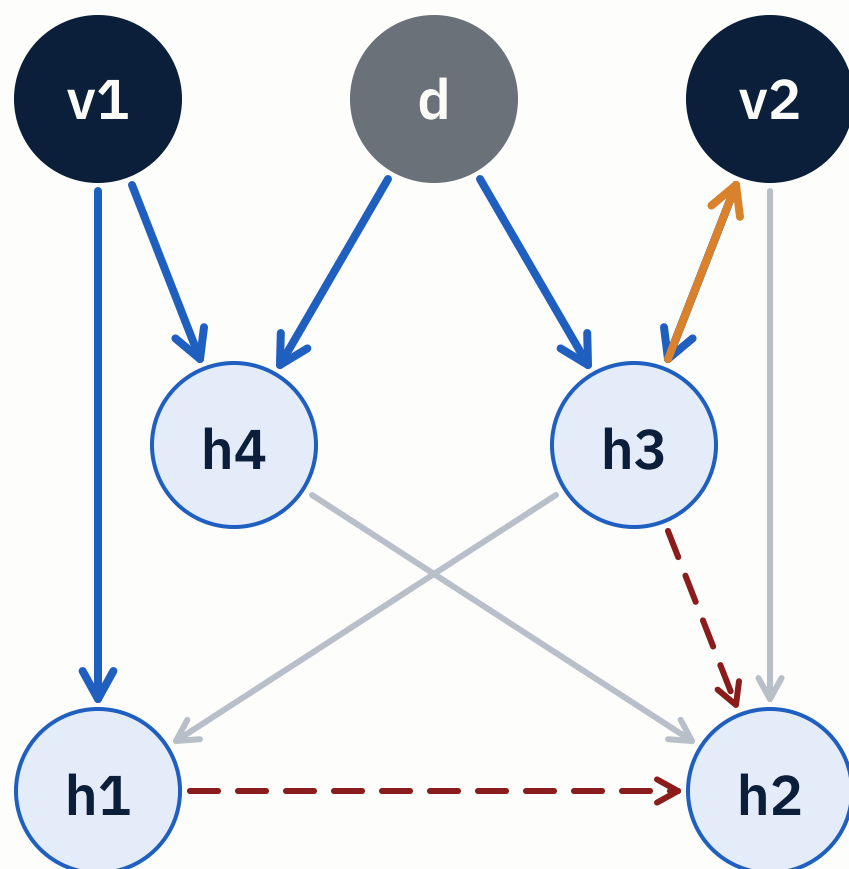
P+ · non-minimum phase



★ P+ is the harder plant to control but the easier one to discover, the opposite of what we predicted. At P+ the valves send most pump flow to the upper tanks ($\gamma_1 + \gamma_2 = 0.77$ vs 1.30), so the drainage links carry a larger share of each lower tank's inflow and leave a stronger trace in the data. Every method gains on exactly those two links.

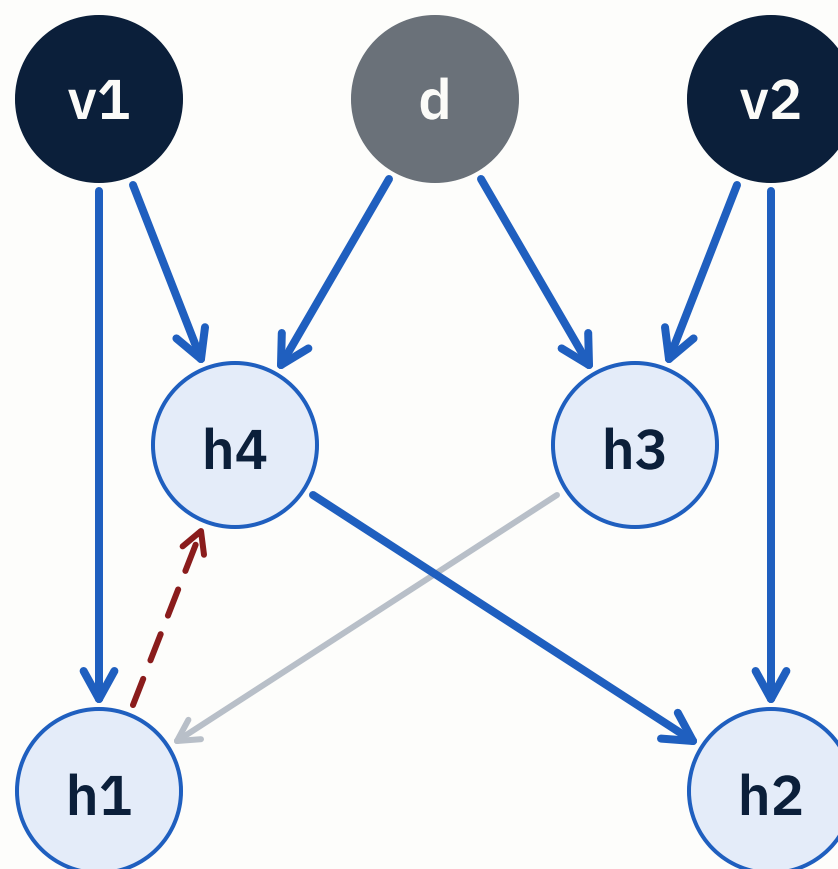
P-: PCMCi+ misses one drainage link; PC and VAR-LiNGAM each lose or flip both

PC (20-step window)



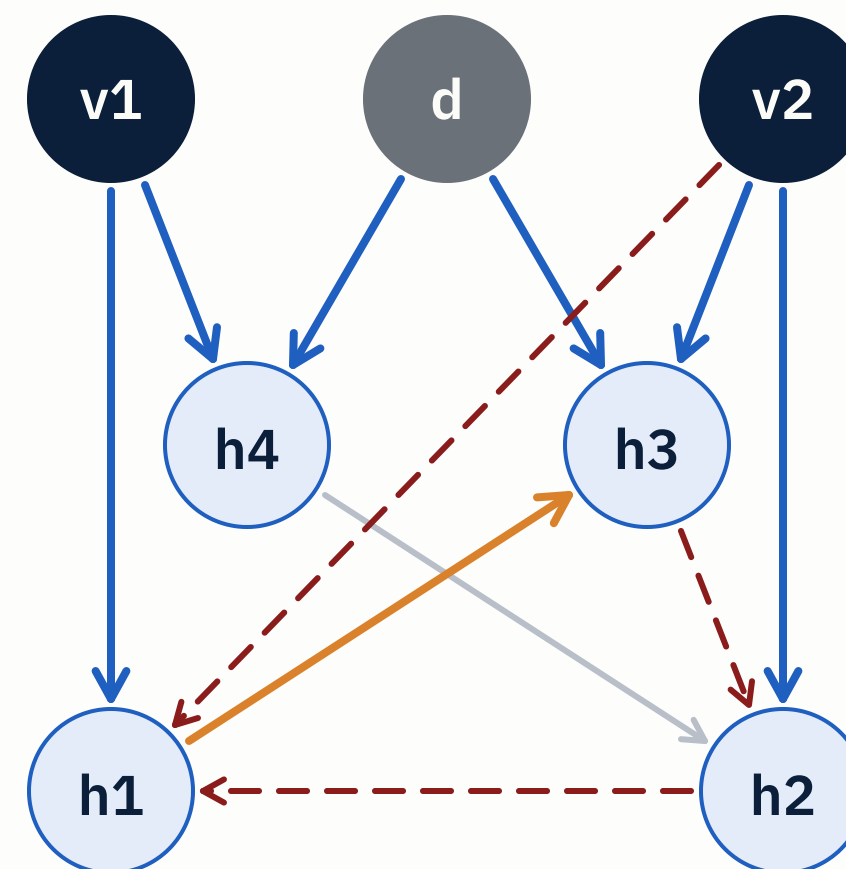
F1 **0.67** · SHD **6** · precision 0.71 · recall 0.62
5 found · 3 missed · 1 reversed · 2 invented

PCMCi+



F1 **0.88** · SHD **2** · precision 0.88 · recall 0.88
7 found · 1 missed · 0 reversed · 1 invented

VAR-LiNGAM



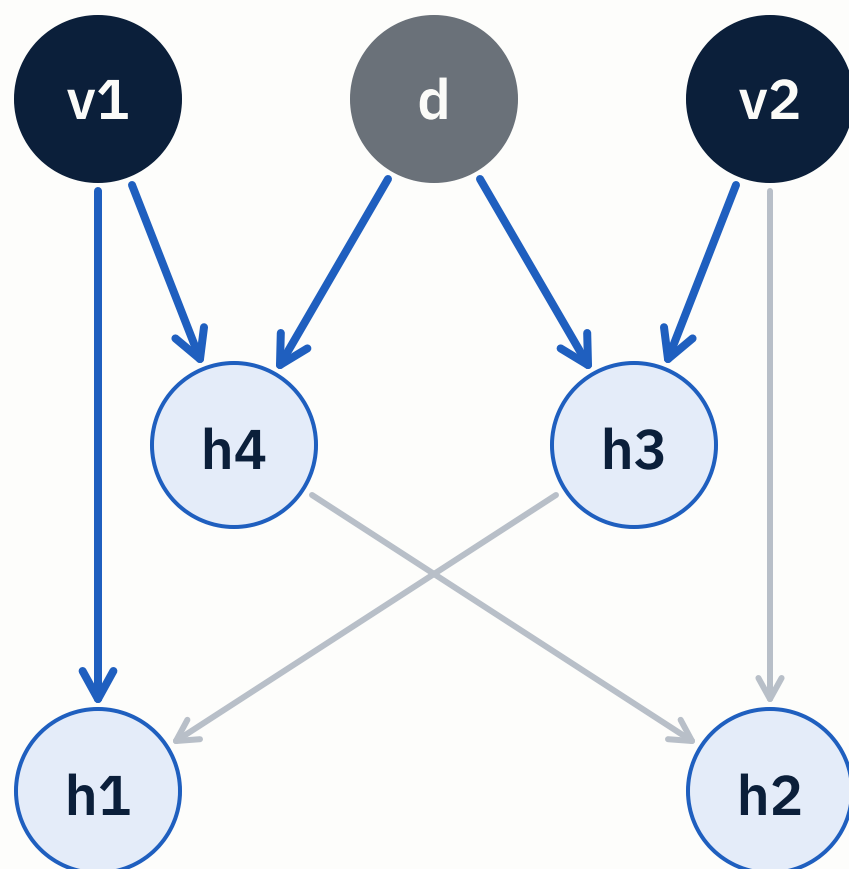
F1 **0.67** · SHD **5** · precision 0.60 · recall 0.75
6 found · 1 missed · 1 reversed · 3 invented

solid blue true link found · **orange** found but pointing the wrong way · **dashed red** invented · **dotted grey** true link missed. Self-loops not drawn.

Source: results/qtank/run-parcorr-tau20-s3/scores.json (edges collapsed over lags)

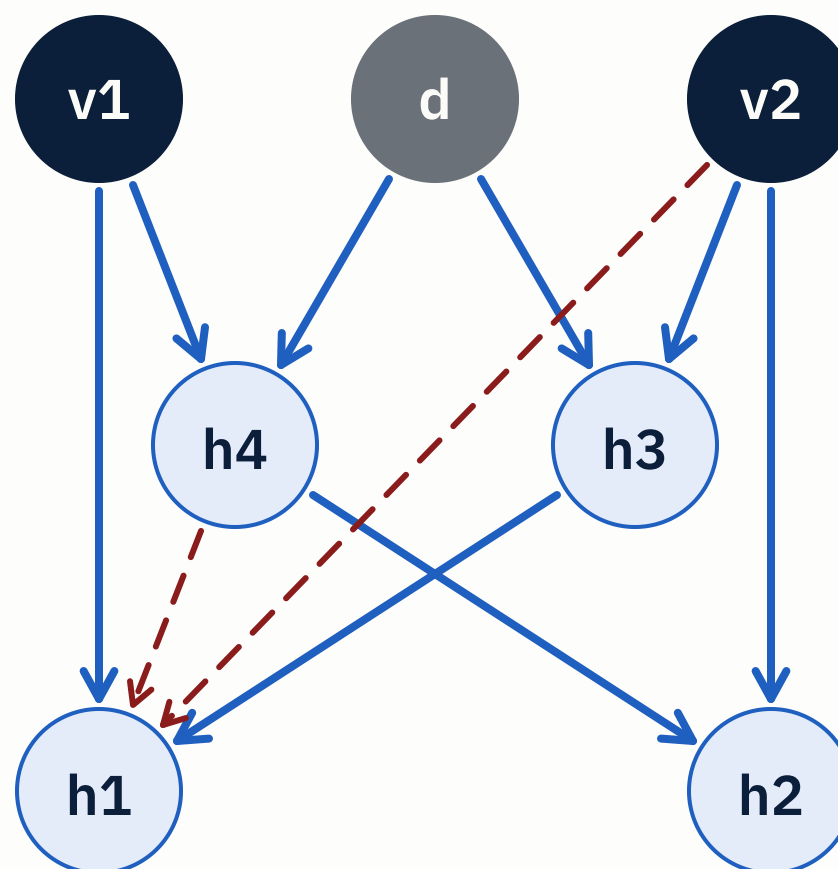
P+: PCMCi+ and VAR-LiNGAM recover all 8 true links; PC finds 5 with nothing invented

PC (20-step window)



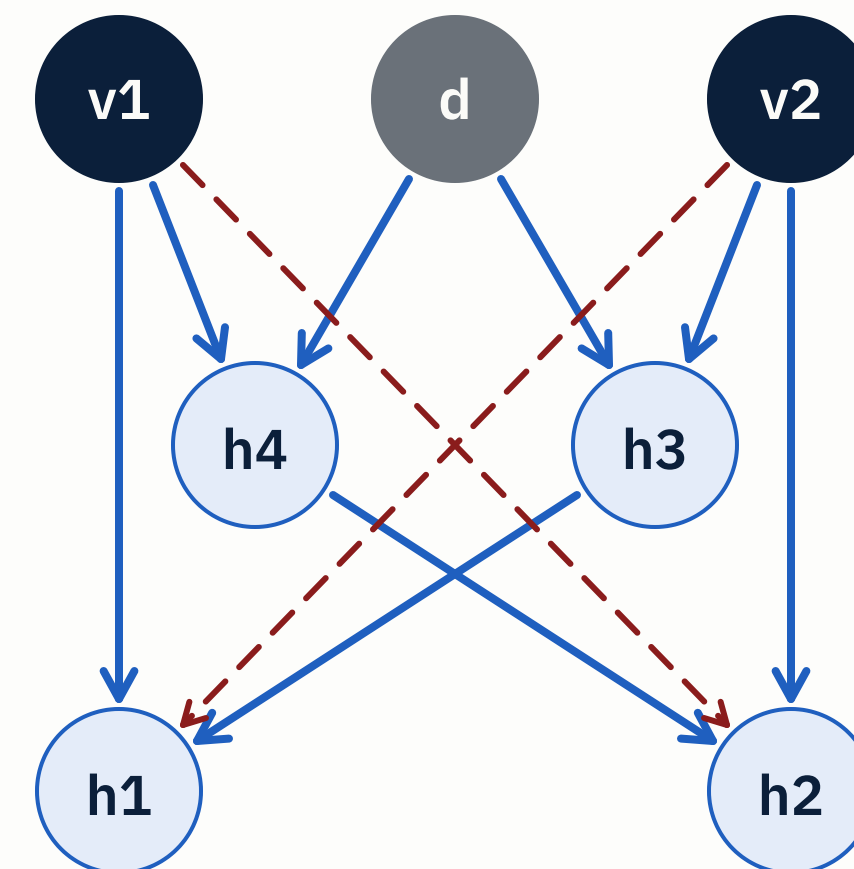
F1 **0.77** · SHD 3 · precision 1.00 · recall 0.62
5 found · 3 missed · 0 reversed · 0 invented

PCMCi+



F1 **0.89** · SHD 2 · precision 0.80 · recall 1.00
8 found · 0 missed · 0 reversed · 2 invented

VAR-LiNGAM



F1 **0.89** · SHD 2 · precision 0.80 · recall 1.00
8 found · 0 missed · 0 reversed · 2 invented

solid blue true link found · **orange** found but pointing the wrong way · **dashed red** invented · **dotted grey** true link missed. Self-loops not drawn.

Source: results/qtank/run-parcorr-tau20-s3/scores.json (edges collapsed over lags)

The two tank-to-tank drainage links are what separate the methods; the six input links are found by everyone

Result for each orange arrow from slide 2, at P- and P+

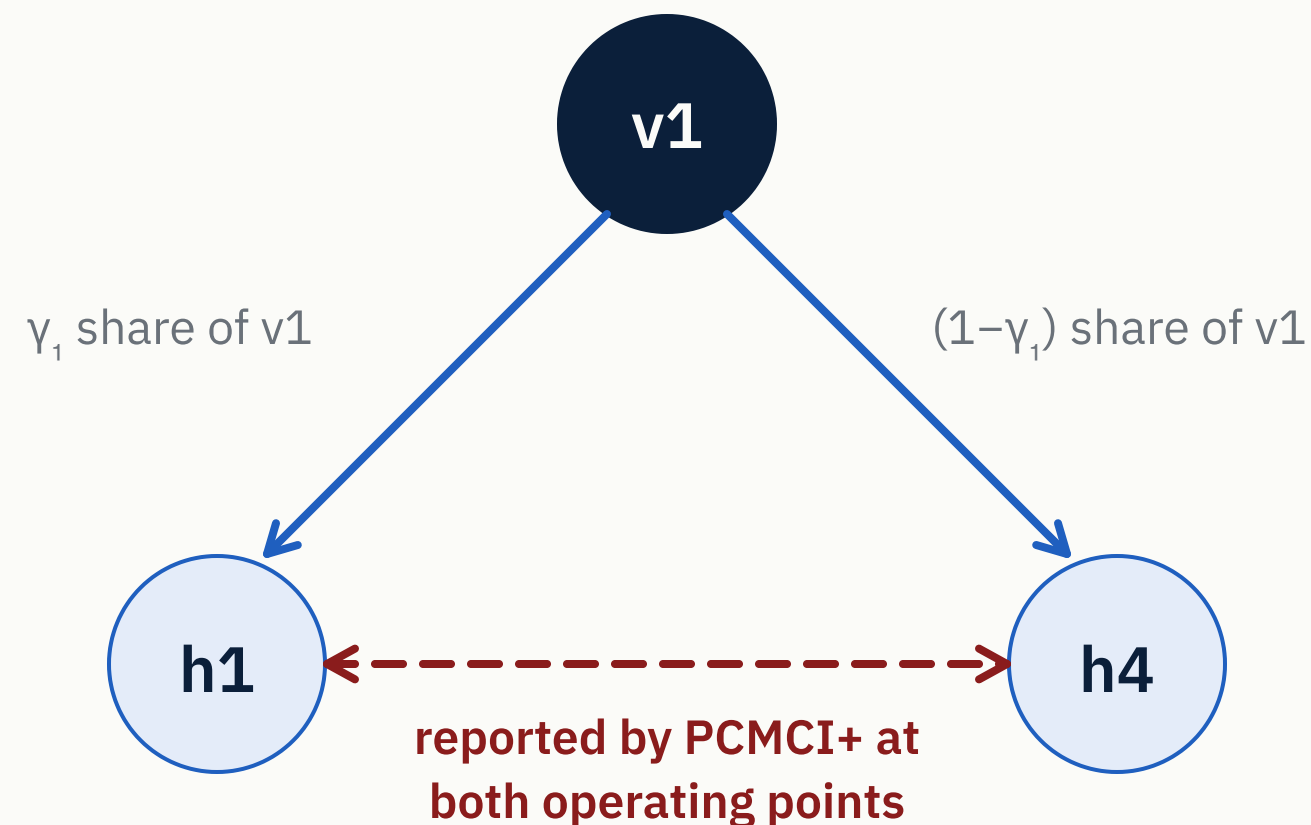
LINK	PC (windowed)	PCMCI+	VAR-LiNGAM
h3 → h1 upper tank 3 drains into lower tank 1	P- missed P+ missed	P- missed P+ found	P- reversed (h1→h3) P+ found
h4 → h2 upper tank 4 drains into lower tank 2	P- missed P+ missed	P- found P+ found	P- missed P+ found

These are the slow, weak links. An upper tank's level is almost fully explained by its own history, which hides its effect on the tank below. Windowed PC also misses one pump link (v2→h2) at both operating points.

THE INVENTED LINKS

All 10 invented links are near misses: a shortcut of a real two-step path, or two tanks fed by the same pump

Correlation between such pairs is expected. A conditional-independence method is supposed to explain it away by conditioning on the shared cause; here it only partly does.



4

Shared-pump siblings: h1–h4 (pump v1) twice, h3–h2 (pump v2) twice. Both tanks receive the same input, in the fixed ratio $\gamma : 1-\gamma$.

4

Shortcuts of a real path: v2→h1 three times, v1→h2 once. The true route is v2→h3→h1; the method skipped the middle tank.

2

Common ancestor: h1–h2, both downstream of the disturbance d through different upper tanks.

Why conditioning does not fully remove them: the levels respond to the pump's history over 60–90 s through a $\sqrt{h}$ outflow, so a linear regression on 20 lagged copies of v1 leaves a residual the two siblings still share. The likely cause, not tested here.

What it means in practice: false alarms land on physically adjacent variables, never on unrelated ones. A root-cause tool built on this will point near the right place even when it is wrong.

BEFORE AND AFTER

Giving plain PC a time window fixes its precision but not its recall

The one methodological change between the two runs: PC scored on the current sample only, then on a 20-step window like the other methods.

PC, no time window			
	Precision	Recall	F1
P−	0.55	0.75	0.63
P+	0.38	0.63	0.48
Invented links: 4 at P−, 6 at P+. Dense, spurious, as pre-registered.			

PC, 20-step window			
	Precision	Recall	F1
P−	0.71	0.63	0.67
P+	1.00	0.63	0.77
Invented links: 2 at P−, 0 at P+. But recall stays at 0.63: it still misses both drainage links and one pump link.			

0.38 → 1.00 precision at P+. The window removes the invented links; it does not find the missing ones.

Source: run-fast/scores.json (tau_max 0, 2880 rows at 5 s) vs run-parcorr-tau20/scores.json (tau_max 20, 960 rows at 15 s). Row count differs too; see the window-length slide.

WINDOW LENGTH

A 300 s window beats a 100 s window for every method, even with a third of the rows

Same 20-lag window, two sampling rates. All 2880 rows at 5 s (window = 100 s, about 1 time constant) vs every 3rd row at 15 s (window = 300 s, about 3 time constants). F1 shown as 100 s $\rightarrow$ 300 s.

PC, windowed

P– **0.57 $\rightarrow$ 0.67**

P+ **0.55 $\rightarrow$ 0.77**

At 100 s it also drops a pump link (v1 $\rightarrow$ h1) at both operating points; at 300 s it gets all four pump links at P+.

PCMCI+

P– **0.82 $\rightarrow$ 0.88**

P+ **0.82 $\rightarrow$ 0.89**

At 100 s it reverses h3 $\rightarrow$ h1 at P+, its only reversal in any run. The longer window removes it and one invented link.

VAR-LiNGAM

P– **0.63 $\rightarrow$ 0.67**

P+ **0.76 $\rightarrow$ 0.89**

Invented links fall from 6 to 3 at P– and 5 to 2 at P+. Runtime halves (300 s $\rightarrow$ 134 s) with 20 lags on fewer rows.

Why: the drainage effect of h3 on h1 builds up over the tank time constants (23–91 s). A 100 s window sees only the start of it; 300 s sees it complete. Rule for plant data: set $\tau_{\max} \cdot T_s$ to at least 3 times the slowest time constant, and subsample rather than shorten.

Source: results/qtank/run-parcorr-tau20-full/scores.json (2880 rows) vs run-parcorr-tau20-s3/scores.json (960 rows); both re-run 28 Sep 2026 with row counts logged

THE SECOND OPINION

VAR-LiNGAM matches PCMCI+ at P+ but pays about 90× the runtime and stumbles on direction at P−

1.5 s

PCMCI+, per dataset

At P+: all 8 true links, 2 invented (F1 0.89)

Ties PCMCI+ on F1 and SHD. The two extras are cross-pump links ($v1 \rightarrow h2$, $v2 \rightarrow h1$).

At P−: one link reversed, one missed (F1 0.67)

Reports $h1 \rightarrow h3$ instead of $h3 \rightarrow h1$ and drops $h4 \rightarrow h2$. Both are drainage links.

134 s

VAR-LiNGAM, per dataset (295–301 s in the earlier run)

Why direction fails on the tanks

Its direction rule needs non-Gaussian noise. The pump signals have it; the tank levels are near-Gaussian because the tanks smooth everything. Predicted before the run.

Where the time goes

100 bootstrap fits with 20 lags each. Not inherent to the estimator; a cheaper setting would cut it.

IMPLICATION

Use PCMCI+ as the default engine for root-cause detection on process data

15 / 16

true links recovered over both operating points; the one miss is $h3 \rightarrow h1$ at P-

0

reversed edges in any run; every reported direction was right

SHD 2

at both operating points; F1 0.88 and 0.89; best or tied-best of the three on every score

1.5 s

per dataset on $960 \text{ rows} \times 7 \text{ variables} \times 20 \text{ lags}$; 90× faster than VAR-LiNGAM

Its three invented links all join variables that share an upstream input, so a wrong answer still points to the right neighbourhood.

The independence test is a plug-in, so the same pipeline extends to nonlinear and non-Gaussian plants without changing the algorithm (next section).

What each method actually assumes about the data, and which assumption breaks first on a real plant

PCMCI+ itself assumes nothing about distributions. The linear-Gaussian assumption sits in the test we plugged into it.

Method	Relationship form	Noise distribution	Breaks when
PC + ParCorr	Linear	Gaussian (for the p-value to be exact)	Nonlinear coupling, e.g. $\sqrt{h}$ outflow far from the operating point
PCMCI+ + ParCorr (this run)	Linear	Gaussian, and independent rows after conditioning on the past	Same as above; autocorrelation is already handled by MCI
PCMCI+ + another test	Set by the test: none for CMIknn	Set by the test: none for CMIknn	Sample size: nonparametric tests need many more rows
VAR-LiNGAM	Linear	Non-Gaussian , required for direction	Gaussian residuals: direction becomes a coin flip (our tank levels)

Here the tanks are near-linear around the operating point and the level noise near-Gaussian, so ParCorr fitted. Valve saturation, on/off logic or heavy-tailed sensor noise on a real plant break both; the fixes follow.

Source: tigramite independence_tests documentation; Runge 2020; Hyvärinen 2010; docs/QTP_EXPECTATIONS.md

For non-Gaussian or nonlinear data, swap the independence test inside PCMCI+; the algorithm stays the same

Every test answers the same question, $X \perp Y \mid S$, with a different statistic. Cost rises as assumptions fall.

ParCorr $\rho(r_x, r_y)$, residuals of linear fits on S Linear, Gaussian. Fastest. Used in this deck. 1.5 s per dataset	RobustParCorr ParCorr on rank-transformed (Gaussianised) data Linear in rank, any marginal. Heavy tails, skew, outliers. Same speed. ~ seconds	GPDC Gaussian-process fit on S, distance correlation of residuals Nonlinear, additive noise. Smooth saturations, $\sqrt{h}$ outflows. Available in our runner. minutes	CMIknn $I(X; Y \mid S)$ from k-nearest-neighbour estimates, shuffle-test p-value No assumptions at all. Needs many rows; 500 shuffles per test. ~67 min per dataset (est.)
---	--	---	---

Our runner already exposes this as a flag (parcorr, gpdc, cmiknn). The CMIknn comparison was configured on 27 September 2026 but has not been run: at 0.4 s per test and ~1000 lagged pairs, that is the 2.7 h in Next steps.

The usual pre-processing fixes make the data look Gaussian but destroy the structure the tests rely on

Log, Box-Cox, quantile transforms

$$y = a \cdot x + e \rightarrow \log y \neq a' \cdot \log x + e'$$

A nonlinear transform of each variable turns a linear causal relation into a nonlinear one. ParCorr then reports a partial dependence that is an artefact of the transform, or misses a real one. The marginals look Gaussian; the joint no longer fits the model.

Differencing, detrending with short filters

$$\Delta h_t = h_t - h_{t-1}$$

The drainage links live in the slow part of the signal (time constants 23–91 s). Differencing at 5 s whitens exactly that band, so the hardest true edges get weaker while high-frequency noise is amplified. Our series are already stationary; there is nothing to remove.

Clipping or winsorising outliers

$$x \leftarrow \min(\max(x, q_{0.01}), q_{0.99})$$

The extreme samples are the informative ones for direction: LiNGAM's identifiability comes entirely from non-Gaussian tails. Trimming them removes the signal it needs and biases the VAR fit near the bounds.

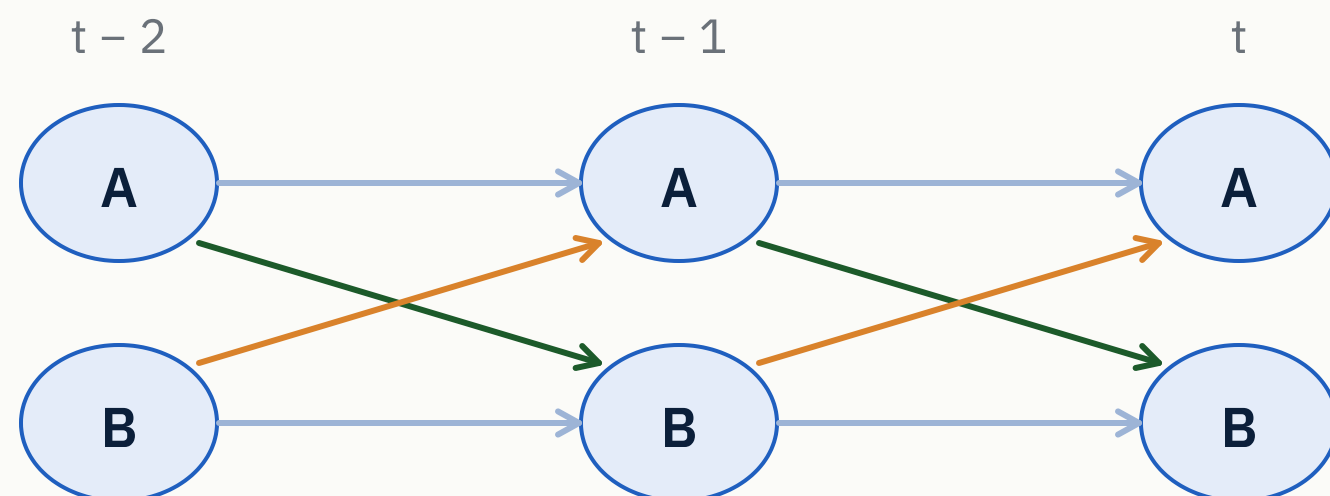
✓ **What is safe:** standardising (z-scoring) each column, because it is linear; removing burn-in; keeping all inputs in the data so no confounder is hidden. Everything else should be handled by choosing the test, not by reshaping the data.

Lagged variables turn feedback loops into an acyclic graph, as long as the loop is slower than the sampling interval

Same-time view: a cycle, no method can orient it



Unrolled in time: a chain, oriented by the clock



$A_{t-1} \rightarrow B_t$ and $B_{t-1} \rightarrow A_t$ are two different edges in the lagged graph. Neither can point backwards in time, so the cycle disappears and the collider rules are never needed for lagged links.

Condition: the loop's round-trip time must be longer than the sampling interval T_s , otherwise both directions collapse into the same-time slice and the cycle returns.

Here: $T_s = 5$ s, tank time constants 23–91 s

5 to 18 samples per loop

Seen in the results: the tanks' self-loops $h_i(t-1) \rightarrow h_i(t)$ are the one cycle this rig has. PCMCi+ reported all 7 self-dependencies (levels and inputs) in both datasets; windowed PC found 6 to 7. The same mechanism will handle recycle streams and control loops on a plant.

NEXT STEPS

Next steps

All three are scoped; no new algorithm code is needed.

What	Cost	What it settles
Run the planned nonlinear-test (CMIknn) comparison for PC and PCMCI+ on both operating points	About 2.7 h compute, config written on 27 Sep 2026	Whether the PCMCI+ win survives a nonlinear independence test
Repeat all runs across several noise seeds	Minutes per seed for PC and PCMCI+; about 2 min per seed for VAR-LiNGAM	Error bars on every F1 in this deck
Run PCMCI+ on real plant data with prior knowledge (known pipes, actuators) supplied as link assumptions	Data access; prior module already exists	Whether the benchmark result transfers to a process with unknown wiring

Thank you

Questions, and the code: every number in this deck traces to a file under results/qtank/ or docs/ in the Root-Cause-Detection repository.